Helion v2

A minimal distributed orchestrator written in Go — built as a vehicle for studying systems programming, distributed systems theory, container orchestration, and production security practices.

v1 (4th semester) demonstrated the core lifecycle: job submission, node registration, heartbeat health checking, crash recovery, and Docker Compose packaging. v2 is a clean-room redesign with a production-grade stack: gRPC, BadgerDB, mTLS, post-quantum cryptography, an Angular dashboard, and Kubernetes-ready deployment (Helm chart + manifests included).

v2 has reached **minimal orchestrator** status with the implementation of all 7 planned features: workflow/DAG support, retry policies, resource-aware scheduling, a full job state machine, priority queues, a real-time event system, and observability improvements (job logs, enhanced metrics, subsystem health checks).

Architecture

```
┌──────────────────────────────────────────┐
│           Angular Dashboard              │
│     REST + WebSocket  ·  JWT auth        │
└──────────────────────────────────────────┘
                   │
                   │ HTTPS
┌──────────────────▼───────────────────────┐
│             Coordinator (Go)             │
│  gRPC Server · Scheduler · BadgerDB      │
│  REST/WS API · Audit Log · Internal CA   │
└──────────────────────────────────────────┘
                   │
                   │ gRPC + mTLS + PQC
          ┌────────┴────────┐
┌─────────▼────────┐  ┌──────▼──────────┐
│  Node Agent      │  │  Node Agent     │
│  Job Executor    │  │  Job Executor   │
│  Log Streamer    │  │  Log Streamer   │
└──────────────────┘  └─────────────────┘
        │                    │
  Runtime interface   Runtime interface
  (Go subprocess + Rust/cgroup/seccomp)
```

The coordinator runs as a Kubernetes `Deployment`. Node agents run as a `DaemonSet` — one per cluster node. The Angular dashboard is served from its own container behind an Nginx reverse proxy.

Stack

Concern	Choice	Why
Primary language	Go 1.26	Native to the K8s/Docker ecosystem; same as etcd, Consul, Prometheus
Runtime	Rust	Memory safety without GC for namespace/cgroup/seccomp code
Inter-node protocol	gRPC + Protocol Buffers	Typed contracts, bidirectional streaming, mTLS-native
Persistence	BadgerDB	Embedded, ACID, pure Go; swap path to etcd if HA is needed

Dashboard	Angular 21	Covers the enterprise framework gap; real WebSocket + auth complexity
Key exchange	ML-KEM / Kyber (NIST FIPS 203)	Hybrid PQC — quantum-resistant from day one
Signatures	ML-DSA / Dilithium (NIST FIPS 204)	Node certificate signing, hybrid with ECDSA
Deployment	Kubernetes + Helm	Cloud-agnostic; one chart, per-cloud values files
CI/CD	GitHub Actions + Snyk	Build, lint, test, coverage gate (≥ 90%), dependency CVE scanning

---

Security model

All coordinator↔node communication is mutually authenticated via mTLS. The coordinator acts as its own internal CA and issues per-node X.509 certificates on first registration. The signed certificate is returned to the node in the `RegisterResponse` so the node presents a coordinator-verified cert on its own gRPC server — the coordinator validates the cert chain during job dispatch.

Key exchange uses a hybrid classical + post-quantum mode (X25519 + ML-KEM/Kyber-768) so sessions are resistant to harvest-now-decrypt-later attacks. Node certificates are signed with ML-DSA (Dilithium) in hybrid mode alongside ECDSA. The coordinator verifies the ML-DSA out-of-band signature on every registration, and pins the cert fingerprint (SHA-256) so a newly-issued cert for the same node ID is rejected unless the node goes through the full revoke → re-register cycle.

The REST API uses short-lived JWTs (15-minute expiry). WebSocket endpoints authenticate via a first-message pattern — the token is sent as the first frame after the handshake, never as a URL query parameter, keeping tokens out of server logs and browser history. The root token is rotated on every coordinator restart — the previous token is immediately revoked so a leaked token from a prior run is dead on restart. Scoped tokens for individual users or services can be issued and revoked via `POST /admin/tokens` and `DELETE /admin/tokens/{jti}` (admin role required). Tokens are stored in memory only — never in `localStorage` or `sessionStorage`. Every job state transition, node registration, auth failure, and token event is written to an append-only audit log in BadgerDB.

Snyk scans Go dependencies and the coordinator container image on every push, blocking on high-severity CVEs. Internal coverage is gated at ≥ 90% on `./internal/...`.

Repository layout

```
helion-v2/
├── cmd/
│   ├── helion-coordinator/   # Coordinator binary
│   ├── helion-node/          # Node agent binary
│   └── helion-run/           # CLI client (submits jobs)
├── internal/
│   ├── api/                  # REST + WebSocket handlers
│   ├── audit/                # Append-only audit logger
│   ├── auth/                 # JWT issuance, CA, certificate management
│   ├── cluster/              # Scheduler, crash recovery, job lifecycle
│   ├── grpcserver/           # gRPC server (coordinator side)
│   ├── grpcclient/           # gRPC client (node agent side)
│   ├── metrics/              # Prometheus metrics provider
│   ├── nodeserver/           # Node agent gRPC server
│   ├── persistence/          # BadgerDB wrapper + key definitions
│   ├── ratelimit/            # Per-node token-bucket rate limiter
│   ├── runtime/              # Runtime interface, Go impl, Rust client
│   └── pqcrypto/             # PQC key generation + hybrid TLS helpers
├── proto/                    # .proto definitions + generated Go stubs
├── dashboard/                # Angular 21 project
```

```
│   └── e2e/                  # Playwright E2E tests (78 tests)
├── deploy/
│   ├── helm/                 # Helm chart
│   ├── k8s/                  # Raw Kubernetes manifests
│   └── docker-compose.yml    # Local dev only
├── runtime-rust/             # Rust runtime binary (cgroup v2 + seccomp)
├── tests/
│   ├── bench/                # Runtime benchmarks (Go vs Rust)
│   └── integration/
│       └── security/         # mTLS, JWT, rate-limit integration tests
├── scripts/
│   └── run-e2e.sh            # One-command full-stack E2E test runner
├── docs/                     # All project documentation — see Documentation index below
├── docker-compose.e2e.yml    # E2E overlay (exposes coordinator HTTP + stable token)
└── .github/workflows/        # GitHub Actions CI/CD
```

Local development

Prerequisites

- Go 1.26+
- Docker + Docker Compose
- `protoc` + `protoc-gen-go` + `protoc-gen-go-grpc` (for protobuf codegen)
- Node.js 20+ (for the Angular dashboard)
- Rust toolchain (optional — only for `runtime-rust/` benchmarks)

Run locally with Docker Compose

```
docker compose up --build
```

This starts one coordinator and two node agents on a shared bridge network. Ports are bound to `127.0.0.1` only.

Build all binaries

```
go build ./...
```

Regenerate protobuf stubs

```
make proto
```

Run tests

```
# All packages — integration tests included
go test -race -count=1 ./...

# Internal packages with coverage report
go test -race -count=1 -coverprofile=coverage.out -covermode=atomic ./internal/...
go tool cover -func=coverage.out | grep total:

# Dashboard unit tests
cd dashboard && ng test --watch=false --browsers=ChromeHeadless
```

```
# Full-stack E2E tests (boots cluster, runs 103 Playwright tests, tears down)
make test-e2e

# E2E with visible browser
make test-e2e-headed

# All test suites in one command (Go + Rust + Angular + E2E)
make test-all
```

CI enforces a ≥ 90% coverage threshold on `./internal/...`.

Environment variables

Variable	Component	Default	Description
`HELION_COORDINATOR`	Node agent, CLI	`localhost:9090`	Coordinator gRPC address
`HELION_ALLOW_ISOLATION`	Node agent	`false`	Enable Linux namespace isolation (requires root or `CAP_SYS_ADMIN`)
`HELION_ROTATE_TOKEN`	Coordinator	`true`	Rotate root token on each startup; `false` reuses the stored token
`HELION_TOKEN_FILE`	Coordinator	`/var/lib/helion/root-token`	Path the rotated root token is written to (mode `0600`)
`HELION_NODE_PINS`	Coordinator	*(unset)*	Pre-configured cert pins `nodeID:sha256hex,…`; unlisted nodes use first-seen
`HELION_DEFAULT_TIMEOUT_SEC`	Runtime	`300`	Fallback job timeout when `TimeoutSeconds` is unset or ≤ 0
`HELION_ALLOWED_COMMANDS`	Runtime	*(unset)*	Comma-separated command allowlist; unset = allow-all (dev mode)
`HELION_CA_CERT_TTL_DAYS`	Coordinator	`730`	Internal CA certificate lifetime
`HELION_NODE_CERT_TTL_HOURS`	Coordinator	`24`	Node certificate lifetime (renewed on each Register)
`HELION_RATE_LIMIT_RPS`	Coordinator	`10`	Per-node job submission rate limit (jobs/second)
`HELION_RUNTIME`	Node agent	`go`	Runtime backend: `go` (subprocess) or `rust` (cgroup v2 + seccomp)
`HELION_RUNTIME_SOCKET`	Node agent	*(unset)*	Unix socket path for Rust runtime
`HELION_TOKEN`	CLI (`helion-run`)	*(unset)*	Bearer token attached to all API requests
`HELION_JOB_ID`	CLI (`helion-run`)	*(unset)*	Stable job ID for idempotent retries
`PORT`	Node agent	`8080`	Node agent listen port

Kubernetes deployment

```
helm install helion ./deploy/helm
```

For cloud-specific values:

```
helm install helion ./deploy/helm -f deploy/helm/values-eks.yaml
helm install helion ./deploy/helm -f deploy/helm/values-gke.yaml
```

The coordinator runs as a single-replica `Deployment`. Node agents run as a `DaemonSet`. HA (multi-coordinator) is a v3 concern.

Known constraints

- Single coordinator replica in v2. HA requires swapping BadgerDB for etcd — the interface is already designed for it.
- Node agents require Linux for namespace isolation. Cross-compiled binaries run on other OS targets without isolation, suitable for local development.
- Namespace isolation requires root or `CAP_SYS_ADMIN`. In Kubernetes this is set in the DaemonSet `SecurityContext`.
- Resource limits (`memory_bytes` , `cpu_quota_us`) are enforced only by the Rust runtime (`HELION_RUNTIME=rust`). The Go runtime ignores them.

---

v1 → v2 changelog

The following bugs from v1 are fixed by design in v2, not patched:

Bug	v1	v2 fix
`CheckHealth()` deadlock	Held write lock during blocking HTTP calls to every node	Replaced with push heartbeat stream — coordinator never polls
`lastIndex` race	Written under `RLock`	Updated via `atomic.Int64`
Double-close on heartbeat channel	`Stop()` called twice	Single ownership, `sync.Once` guard
State round-trip of `*PersistentState`	Serialised pointer to its own path, silent nil on reload	State excluded from JSON; BadgerDB handles persistence
Crash recovery timing	`recoverLostJobs()` fired before nodes re-registered	15 s startup grace period before dispatch is attempted

---

Documentation index

All project documentation lives under `docs/` :

File	Contents
[README.md]()	This file — project overview, stack, environment variables, quick start
[ARCHITECTURE.md]()	High-level design, technology decisions, protocol contracts, CI/CD
[COMPONENTS.md]()	Coordinator, node agent, and runtime interface internals
[PERFORMANCE.md]()	Go vs Rust benchmarks, cgroup/seccomp overhead
[SECURITY.md]()	Threat model, mTLS, PQC, rate limiting, audit logging
[JWT-GUIDE.md]()	JWT token lifecycle, issuance, usage, revocation

SECURITY-OPS.md	Operational checklist, env vars, troubleshooting
AUDIT.md	Security & code-quality audit **template** — copy into `audits/<YYYY-MM-DD>.md` to start a new audit
audits/	Archive of closed audits (one file per run, filename = audit ID)
dashboard.md	Angular dashboard — stack, testing, local dev
persistence.md	`internal/persistence` rules, key schema, test invariants
docker-compose-dev-notes.md	Local Docker Compose workflow notes
planned-features/	Feature specs for orchestrator enhancements

Packing the repo without audits and planned features

`docs/audits/` and `docs/planned-features/` grow over time. To bundle the source tree for download, an AI assistant, or offline review **without** those directories:

```
# With repomix:
npx repomix --ignore "docs/audits/**,docs/planned-features/**"

# Or with git:
git archive --format=tar.gz -o helion-v2.tar.gz HEAD \
  -- ':(exclude)docs/audits' ':(exclude)docs/planned-features'
```

This keeps `docs/AUDIT.md` (the template) and all other docs, dropping only the dated audit files and feature spec archive.

Dennis Alves Pedersen · April 2026

Further reading: Architecture reference · Security reference